Building data course — Week 2 — units, joins and conditions

sparql-playground / lessons

Generated for print and offline reading

Week 2 — units, joins and conditions

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. Each lesson's two scripts are printed after it. Keep notes under the ignored `student-work/` folder.

Contents

- Day 6 — Units and SI: convert once, then do all fault math in °C, Pa and fractions
- Day 7 — Two AHUs: stack them, then join them on time
- Day 8 — CASE and conditional aggregation: pressure while running, hours while unoccupied
- Day 9 — Time buckets: hourly and daily averages, and views vs summary tables
- Day 10 — Joining Open-FDD's results: episodes per rule, then tag every row with its fault episode

---

Day 6 — Units and SI: convert once, then do all fault math in °C, Pa and fractions

Goal: turn the BAS's US-unit columns into SI columns (a SQL view, a pandas `assign`), and avoid the one conversion mistake everybody makes: converting a temperature *difference*.

Read

The CSV is what the building automation system (BAS) logged: temperatures in °F, pressures in inches of water (inH2O), valve and fan speeds in percent 0–100. **Open-FDD's fault math is SI-only**: °C (or K for differences), pascals, and fractions 0–1. There are two reasons:

1. **One set of thresholds.** Rules ship with numbers like "25 Pa" or "2 K". If every site could feed °F one day and °C the next, every rule would need a unit switch. Instead the *ingest* step converts, and the rules never see US units.
2. **Fewer silent bugs.** A rule that compares `percent` to `0.95` is wrong by 100× and never throws an error. If everything is a fraction, "95%" is `0.95` everywhere.

So the first step of every rule is a conversion. You write it **once**, in one place (a view in SQL, a function or `assign` in pandas), and every later day reuses it.

Quantity	CSV unit → SI	Formula
temperature (absolute)	°F → °C	`(F - 32) * 5 / 9`
temperature (difference)	°F → K	`F * 5 / 9` (**no 32**)
static pressure	inH2O → Pa	`inwc * 249.08891`
valve, damper, fan speed	percent → fraction	`pct / 100`

The delta trap. The 32 in `(F - 32) * 5 / 9` exists because 0 °F and 0 °C are different *places* on the scale. A *difference* has no zero point to shift: a 5 °F rise is a 5 × 5/9 = **2.78 K** rise. Push it through the absolute formula and you get `(5 - 32) * 5 / 9 = -15`, a cooling of 15 degrees. It's a typo-sized mistake that turns a "DAT is 5 °F over setpoint" check into nonsense. The scripts build the discharge-air-temperature error three ways: convert the difference (right), subtract two converted temperatures (right, same numbers), and the wrong one.

Conversions of this kind are linear, so the mean of the converted column equals the converted mean. That's a cheap sanity check, but only for linear conversions.

Step	`day06_sql.py`	`day06_pandas.py`
load	`FROM ahu_1`	`pd.read_csv(..., usecols=COLUMNS)`
convert	`CREATE TEMP VIEW ahu_1_si AS SELECT (x - 32) * 5 / 9.0 AS ...`	`raw.assign(x_c=(raw.x_f - 32) * 5 / 9)`
percent → fraction	`supply_fan_speed_pct / 100.0`	`raw.supply_fan_speed_pct / 100`
delta	`(a_f - b_f) * 5 / 9.0`	`(raw.a_f - raw.b_f) * 5 / 9`

Step	day06_sql.py	day06_pandas.py
mean of every column	one `avg()` per column, `UNION ALL`	`si.mean()`

Watch the `/ 9.0` and `/ 100.0` in SQL: SQLite divides integers as integers (`5 / 9` is `0`), so one operand must be a float. pandas always does true division.

Run

```
uv run python lessons/course.py run 6
```

That runs `day06_pandas.py`, then `day06_sql.py`. Both print:

```
A 5 degF *difference*:
 delta_f  right_k  wrong_k
     5.0     2.78    -15.0
...
Mean of each SI column:
               column    mean
 discharge_air_temp_c  16.206
       dat_setpoint_c   9.424
   outside_air_temp_c  16.265
       duct_static_pa 195.035
    duct_static_sp_pa 340.226
supply_fan_speed_frac   0.390
       chw_valve_frac   0.189
```

A SQL view stores the query, not the rows: every `SELECT` from `ahu_1_si` re-runs the conversion. That costs nothing for arithmetic on 35,536 rows, and it means the view can't go stale. Day 9 compares views with summary tables.

Try it

1. Add a column in both scripts for the mixed-air temperature, `mixed_air_temp_f` → °C. Is the mean what you expect next to the discharge and outside air means?
2. A rule says "fault if discharge air is more than 5 K above its setpoint". Which of `dat_error_k` and `dat_error_wrong` do you compare with 5? How many rows does each flag? And if the threshold were written as 9 °F, what do you compare against?

Answers

1. SQL: `(mixed_air_temp_f - 32) * 5 / 9.0 AS mixed_air_temp_c`; pandas: `mixed_air_temp_c=(raw["mixed_air_temp_f"] - 32. ▪ 5 / 9. Remember to add the column tousecols`.
2. `dat_error_k > 5` flags 16,488 rows (about 46% of the history). The wrong column flags only 2,058, because it is shifted by about -17 K. A 9 °F

threshold is itself a *difference*, so convert it as `9 * 5 / 9 = 5 K`: multiply, don't subtract 32.

Why it matters

Every fault threshold from here on is written in SI, so every script from here on starts with the conversion you wrote today. If the conversion is wrong, every fault count downstream is wrong in a way no test on the *rule* will catch. Week 5 moves the unit into the data model (each point says what unit it logs), so a rule can ask for "duct static in Pa" and the conversion is looked up, not typed.

day06_pandas.py

```
"""Day 6, pandas: convert AHU_1's FDD columns from US units to SI
        (degC, Pa, fractions 0-1).

    uv run python lessons/unit1-sqlite/day06_pandas.py

Fault math is SI-only, like Open-FDD's. The BAS logged degF, inches of
        water and percent 0-100, so
the first step of every rule is a conversion. `assign` adds the
        converted columns next to the raw
ones.
Compare the printout with day06_sql.py: the numbers should match.
"""

from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

PA_PER_INH2O = 249.08891  # 1 inch of water column = 249.08891 pascals

# 1. Load: only the columns we convert
COLUMNS = [
    "timestamp_utc",
    "discharge_air_temp_f",
    "dat_reset_f",
    "outside_air_temp_f",
    "da_p_inwc",
    "da_p_setpoint_inwc",
    "supply_fan_speed_pct",
    "chw_valve_pct",
]
raw = pd.read_csv(DATA / "AHU_1" / "history_wide.csv", usecols=COLUMNS)
        [COLUMNS]

# 2. Convert. Absolute temperatures use (F - 32) * 5/9. Pressure is a
        plain factor. Percent is /
#    100.
si = raw[["timestamp_utc"]].assign(
    discharge_air_temp_c=(raw["discharge_air_temp_f"] - 32) * 5 / 9,
    dat_setpoint_c=(raw["dat_reset_f"] - 32) * 5 / 9,
    outside_air_temp_c=(raw["outside_air_temp_f"] - 32) * 5 / 9,
    duct_static_pa=raw["da_p_inwc"] * PA_PER_INH2O,
    duct_static_sp_pa=raw["da_p_setpoint_inwc"] * PA_PER_INH2O,
    supply_fan_speed_frac=raw["supply_fan_speed_pct"] / 100,
    chw_valve_frac=raw["chw_valve_pct"] / 100,
)

# 3. The delta trap: DAT error is a temperature DIFFERENCE. A
        difference has no 32 in it: x degF =
#    x * 5/9 K.
#    Right: convert the difference, or subtract two converted values
        (same answer).
#    Wrong: push the difference through the absolute formula.
delta_f = raw["discharge_air_temp_f"] - raw["dat_reset_f"]
```

```
deltas = raw[["timestamp_utc"]].assign(
    dat_error_k=delta_f * 5 / 9,
    dat_error_k_check=si["discharge_air_temp_c"] -
         si["dat_setpoint_c"],
    dat_error_wrong=(delta_f - 32) * 5 / 9,
)

# 4. Summaries: means in SI, and the 5 degF demo
means = si.drop(columns="timestamp_utc").mean().to_frame("mean").rename_axis("co
five_f = pd.DataFrame({"delta_f": [5.0], "right_k": [5 * 5 / 9],
          "wrong_k": [(5 - 32) * 5 / 9]})

RESULTS = {"ahu_1_si": si, "dat_error": deltas, "si_means": means,
          "five_f": five_f}

print("A 5 degF *difference*:")
print(five_f.round(2).to_string(index=False))
print("\nFirst rows, US units then SI:")
print(raw.head(3).round(2).to_string(index=False))
print(si.head(3).round(2).to_string(index=False))
print("\nMean of each SI column:")
print(means.round(3).to_string(index=False))
print("\nDAT error (K), right vs wrong, over all rows:")
print(deltas[["dat_error_k",
          "dat_error_wrong"]].describe().loc[["mean", "min",
          "max"]].round(2))
```

day06_sql.py

```
"""Day 6, SQL: convert AHU_1's FDD columns from US units to SI (degC,
        Pa, fractions 0-1).

    uv run python lessons/course.py setup         # once: loads the
        CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day06_sql.py

The same conversions as day06_pandas.py, as a TEMP VIEW: a saved SELECT
        that every later query can
use as if it were a table. SQLite does all the work; pandas only turns
        the result rows into a
table for printing.
Compare the printout with day06_pandas.py: the numbers should match.
"""

import os
import sqlite3
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {"pa_per_inh2o": 249.08891}  # 1 inch of water column =
        249.08891 pascals

# 1 + 2. Load and convert in one view. ahu_1 is the CSV loaded as a
        table, same column names.
#        Absolute temperatures use (F - 32) * 5/9. Pressure is a plain
        factor. Percent is / 100.
SI_VIEW_SQL = """
CREATE TEMP VIEW ahu_1_si AS
SELECT timestamp_utc,
       (discharge_air_temp_f - 32) * 5 / 9.0 AS discharge_air_temp_c,
       (dat_reset_f - 32) * 5 / 9.0          AS dat_setpoint_c,
       (outside_air_temp_f - 32) * 5 / 9.0   AS outside_air_temp_c,
       da_p_inwc * :pa_per_inh2o             AS duct_static_pa,
       da_p_setpoint_inwc * :pa_per_inh2o    AS duct_static_sp_pa,
       supply_fan_speed_pct / 100.0          AS supply_fan_speed_frac,
       chw_valve_pct / 100.0                 AS chw_valve_frac
FROM ahu_1
"""
# SQLite can't bind parameters inside CREATE VIEW, so put the number in
        the text (it's a constant).
conn.execute(SI_VIEW_SQL.replace(":pa_per_inh2o",
        repr(PARAMS["pa_per_inh2o"])))

# 3. The delta trap: DAT error is a temperature DIFFERENCE. A
        difference has no 32 in it: x degF =
#    x * 5/9 K.
```

```
#    Right: convert the difference, or subtract two converted values
         (same answer).
#    Wrong: push the difference through the absolute formula.
DELTA_SQL = """
SELECT timestamp_utc,
       (discharge_air_temp_f - dat_reset_f) * 5 / 9.0        AS
         dat_error_k,
       (SELECT discharge_air_temp_c - dat_setpoint_c FROM ahu_1_si s
        WHERE s.timestamp_utc = a.timestamp_utc)              AS
         dat_error_k_check,
       ((discharge_air_temp_f - dat_reset_f) - 32) * 5 / 9.0 AS
         dat_error_wrong
FROM ahu_1 a
ORDER BY timestamp_utc
"""

# 4. Summaries: means in SI, and the 5 degF demo
MEANS_SQL = """
SELECT 'discharge_air_temp_c' AS "column", avg(discharge_air_temp_c) AS
         mean FROM ahu_1_si UNION ALL
SELECT 'dat_setpoint_c', avg(dat_setpoint_c) FROM ahu_1_si UNION ALL
SELECT 'outside_air_temp_c', avg(outside_air_temp_c) FROM ahu_1_si
         UNION ALL
SELECT 'duct_static_pa', avg(duct_static_pa) FROM ahu_1_si UNION ALL
SELECT 'duct_static_sp_pa', avg(duct_static_sp_pa) FROM ahu_1_si UNION
         ALL
SELECT 'supply_fan_speed_frac', avg(supply_fan_speed_frac) FROM
         ahu_1_si UNION ALL
SELECT 'chw_valve_frac', avg(chw_valve_frac) FROM ahu_1_si
"""
FIVE_F_SQL = "SELECT 5.0 AS delta_f, 5 * 5 / 9.0 AS right_k, (5 - 32) *
         5 / 9.0 AS wrong_k"

si = pd.read_sql_query("SELECT * FROM ahu_1_si ORDER BY timestamp_utc",
         conn)
deltas = pd.read_sql_query(DELTA_SQL, conn)
means = pd.read_sql_query(MEANS_SQL, conn)
five_f = pd.read_sql_query(FIVE_F_SQL, conn)
raw = pd.read_sql_query("SELECT * FROM ahu_1 ORDER BY timestamp_utc
         LIMIT 3", conn)

RESULTS = {"ahu_1_si": si, "dat_error": deltas, "si_means": means,
         "five_f": five_f}

print("A 5 degF *difference*:")
print(five_f.round(2).to_string(index=False))
print("\nFirst rows, US units then SI:")
print(
    raw[
        [
            "timestamp_utc",
            "discharge_air_temp_f",
            "dat_reset_f",
            "outside_air_temp_f",
            "da_p_inwc",
            "da_p_setpoint_inwc",
            "supply_fan_speed_pct",
            "chw_valve_pct",
        ]
```

```
    ]
    .round(2)
    .to_string(index=False)
)
print(si.head(3).round(2).to_string(index=False))
print("\nMean of each SI column:")
print(means.round(3).to_string(index=False))
print("\nDAT error (K), right vs wrong, over all rows:")
print(deltas[["dat_error_k",
         "dat_error_wrong"]].describe().loc[["mean", "min",
         "max"]].round(2))
```

Day 7 — Two AHUs: stack them, then join them on time

Goal: put both AHUs in one table, then JOIN them on `timestamp_utc` to ask "how far apart are two sensors that measure the same weather?", in SQL and in pandas, and learn how a join actually runs and how it goes wrong.

Read

Building 50 has two air handlers, AHU_1 and AHU_2, each logged in its own CSV (and its own SQLite table) with the same column names. Two different questions need two different tools.

Stack: "give me all the rows." One table with an `equipment_id` column, so one query covers every AHU. This is `UNION ALL` in SQL and `pd.concat` in pandas (add the id column to each piece first). Use `UNION ALL`, not `UNION`: plain `UNION` also removes duplicate rows, which costs a sort and would silently drop legitimate repeats.

Join: "put the two rows for the same time side by side." One row per timestamp, one column per AHU. Both AHUs sit in the same building and see the same weather, so their outside-air temperatures should nearly agree. A big gap means one sensor is wrong, a better data-quality check than any threshold on a single sensor. The difference of two converted temperatures is already a *difference* (Day 6): `oa_ahu_1_c - oa_ahu_2_c` is in K, no 32.

Building 50's answer: on average the sensors differ by 1.6 K, 9,005 of 35,536 rows by more than 2 K, and 7 rows by more than 20 K. Those 7 are AHU_2 reading exactly 0 °F, a sensor dropout: the history's minimum for AHU_2 is −17.8 °C, about 10 K colder than AHU_1's.

How a join runs. Three ways to match the rows:

Algorithm	How	Cost (n rows probing m rows)	Where you meet it
nested loop	for each left row, scan the whole right table	O(n · m)	a naive double loop
nested loop + index seek	for each left row, look the key up in a B-tree	O(n log m)	SQLite, using the primary key
hash join	build a hash table of the right side once, probe it per left row	O(n + m)	pandas `merge`

`timestamp_utc` is the table's primary key, so SQLite already has a B-tree on it, and the plan says `SEARCH b USING COVERING INDEX sqlite_autoindex_ahu_2_1 (timestamp_utc=?)`. Without an index SQLite doesn't fall back to the O(n·m) loop: it builds a temporary **automatic index** just for the query. pandas builds a hash table. The scripts print a timing at the end to show the difference.

Join explosion. The one-to-one join above works because `timestamp_utc` is unique in each table. If the join key repeats, every left row pairs with *every* matching right row. Try joining the first day of the two AHUs on the **hour** (about 12 rows per hour) instead of the timestamp: 280 rows per side become 3,328 rows. Add up a `SUM` or `COUNT` over a join like that and every number is silently inflated. pandas has a guard: `merge(..., validate="1:1")` raises `MergeError` if either side's key repeats (`"1:m"` and `"m:1"` check one side). SQL has no such keyword; the equivalent is to check `count(*) > count(DISTINCT key)` before you trust a join.

Step	`day07_sql.py`	`day07_pandas.py`
stack	`SELECT 'AHU_1' AS equipment_id, ... UNION ALL SELECT 'AHU_2', ...`	`pd.concat(frames)` after `df.insert(0, "equipment_id", ...)`
join on time	`FROM ahu_1 a JOIN ahu_2 b ON b.timestamp_utc = a.timestamp_utc`	`a.merge(b, on="timestamp_utc", validate="1:1")`
gap stats	`avg(abs(diff_k)), sum(abs(diff_k) > 2)`	`diff.abs().mean(), (diff.abs() > 2).sum()`
key repeats?	`count(*) > count(DISTINCT hour)`	`merge(..., validate="1:1")` raises

Run

```
uv run python lessons/course.py run 7
```

That runs `day07_pandas.py`, then `day07_sql.py`. Both print:

```
Outside-air temperature, AHU_1 minus AHU_2 (K):
 paired_rows  mean_abs_diff_k  max_abs_diff_k  rows_over_2k
rows_over_20k
       35536             1.64           39.83          9005
7

Join explosion on one day of data:
 rows_per_side  joined_on_timestamp  joined_on_hour  validate_blocked
           280                  280            3328              True
```

(SQLite prints `1` instead of `True`; the test treats them as equal.) After that, the pandas script times a nested loop against `merge` (1,500 rows per side: tens of milliseconds against about 1 ms) and the SQL script prints the two query

plans and times them. Your times will differ; the shape won't.

Try it

1. Change the 2 K threshold to 1 K in both scripts. How many rows are more than 1 K apart?
2. Make it a left join (`LEFT JOIN` / `how="left"`) after deleting one AHU_2 row (in pandas: `b = b.drop(b.index[100])`). What is `oa_ahu_2_c` on the missing timestamp, and how does that differ from the inner join's result?

Answers

1. Compare `abs(diff_k) > 1`: 17,215 rows, just under half. The two sensors disagree by more than 1 K for much of the record, so a rule that mixes them shouldn't assume they match.
2. The left join keeps the AHU_1 row and fills the AHU_2 value with `NaN` (SQL: `NULL`); the inner join silently drops the row, so you would never know a timestamp was missing. Left joins are how you *find* gaps.

Why it matters

Stacking and joining are the two shapes of nearly every analysis: "same question for every AHU" and "relate two measurements at the same time". Both are exact and cheap *when the key is unique and you know it is*. The join explosion is the classic way an energy or fault number comes out 12× too big and nobody notices. Day 8 adds `CASE`, so one pass over the rows can answer several conditional questions at once.

day07_pandas.py

```python
"""Day 7, pandas: stack two AHUs, then join them on time to compare
        their outside-air sensors.

    uv run python lessons/unit1-sqlite/day07_pandas.py

Both AHUs measure the same weather, so their outside-air temperatures
        should nearly agree.
`concat` stacks the rows (one table, an equipment_id column); `merge`
        lines them up side by side
by timestamp.
Compare the printout with day07_sql.py: the numbers should match.
"""

import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

# 1. Load + stack: one CSV per AHU, an equipment_id column says which
        rows came from where (UNION
#    ALL)
COLUMNS = ["timestamp_utc", "outside_air_temp_f"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip / "history_wide.csv",
        usecols=COLUMNS)
    df.insert(0, "equipment_id", equip)
    frames.append(df)
stacked = pd.concat(frames, ignore_index=True)

# 2. SI units (Day 6)
stacked["outside_air_temp_c"] = (stacked["outside_air_temp_f"] - 32) *
        5 / 9

# 3. What the stacked table holds, per AHU
stack_summary = (
    stacked.groupby("equipment_id")
    .agg(
        rows=("timestamp_utc", "size"),
        first_ts=("timestamp_utc", "min"),
        last_ts=("timestamp_utc", "max"),
        mean_oa_c=("outside_air_temp_c", "mean"),
        min_oa_c=("outside_air_temp_c", "min"),
    )
    .reset_index()
)

# 4. Join on timestamp: one row per timestamp, one column per AHU (an
        inner join, 1:1).
#    validate="1:1" makes pandas raise if either side has a repeated
        timestamp.
a = stacked[stacked.equipment_id == "AHU_1"][["timestamp_utc",
        "outside_air_temp_c"]]
```

```
b = stacked[stacked.equipment_id == "AHU_2"][["timestamp_utc",
        "outside_air_temp_c"]]
pairs = a.merge(b, on="timestamp_utc", suffixes=("_ahu_1", "_ahu_2"),
        validate="1:1")
pairs = pairs.rename(
    columns={"outside_air_temp_c_ahu_1": "oa_ahu_1_c",
        "outside_air_temp_c_ahu_2": "oa_ahu_2_c"}
)
# a difference of two converted temperatures is already in kelvin (Day
        6's delta rule)
pairs["diff_k"] = pairs["oa_ahu_1_c"] - pairs["oa_ahu_2_c"]

# 5. How far apart are they?
absd = pairs["diff_k"].abs()
gap = pd.DataFrame(
    {
        "paired_rows": [len(pairs)],
        "mean_abs_diff_k": [absd.mean()],
        "max_abs_diff_k": [absd.max()],
        "rows_over_2k": [int((absd > 2).sum())],
        "rows_over_20k": [int((absd > 20).sum())],
    }
)

# 6. Join explosion: the first day (280 rows per AHU), joined on the
        HOUR instead of the timestamp.
#    The hour repeats 12 times per AHU, so every row matches 12 rows on
        the other side.
day_a = a[a.timestamp_utc < "2026-03-17"].assign(hour=lambda d:
        d.timestamp_utc.str[:13])
day_b = b[b.timestamp_utc < "2026-03-17"].assign(hour=lambda d:
        d.timestamp_utc.str[:13])
by_ts = day_a.merge(day_b, on="timestamp_utc")
by_hour = day_a.merge(day_b, on="hour")
try:
    day_a.merge(day_b, on="hour", validate="1:1")
    blocked = False
except pd.errors.MergeError:
    blocked = True
explosion = pd.DataFrame(
    {
        "rows_per_side": [len(day_a)],
        "joined_on_timestamp": [len(by_ts)],
        "joined_on_hour": [len(by_hour)],
        "validate_blocked": [blocked],
    }
)

RESULTS = {
    "stack_summary": stack_summary,
    "oa_pairs": pairs,
    "oa_gap": gap,
    "explosion": explosion,
}

print("Stacked table, per AHU:")
print(stack_summary.round(2).to_string(index=False))
print("\nOutside-air temperature, AHU_1 minus AHU_2 (K):")
```

```
print(gap.round(2).to_string(index=False))
print("\nJoin explosion on one day of data:")
print(explosion.to_string(index=False))

# Timing (not part of RESULTS): hash join (merge) vs a hand-written
         nested loop on 1,500 rows per
#    side.
n = 1500
left = a.head(n)["timestamp_utc"].tolist()
right = b.head(n)["timestamp_utc"].tolist()
t0 = time.perf_counter()
loops = sum(1 for x in left for y in right if x == y)
t_loop = time.perf_counter() - t0
t0 = time.perf_counter()
merged = a.head(n).merge(b.head(n), on="timestamp_utc")
t_merge = time.perf_counter() - t0
t0 = time.perf_counter()
a.merge(b, on="timestamp_utc", validate="1:1")
t_full = time.perf_counter() - t0
print(
    f"\nnested loop, {n} x {n} rows: {t_loop * 1000:8.1f} ms  "
    f"({loops} matches, {n * n:,} comparisons)"
)
print(f"merge (hash join), same:    {t_merge * 1000:8.1f} ms
         ({len(merged)} matches)")
print(f"merge, all {len(a):,} rows:   {t_full * 1000:8.1f} ms")
```

day07_sql.py

```
"""Day 7, SQL: stack two AHUs, then join them on time to compare their
        outside-air sensors.

    uv run python lessons/course.py setup         # once: loads the
        CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day07_sql.py

The same computation as day07_pandas.py, done by SQLite on the same
        rows. SQLite does all the
work; pandas only turns the result rows into a table for printing.
Compare the printout with day07_pandas.py: the numbers should match.
"""

import os
import sqlite3
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {"day_end": "2026-03-17"}  # the explosion demo uses the first
        day only

# 1 + 2. Stack (UNION ALL) and convert to SI (Day 6). ahu_1 and ahu_2
        are the CSVs loaded as tables.
STACK_VIEW_SQL = """
CREATE TEMP VIEW stacked AS
SELECT 'AHU_1' AS equipment_id, timestamp_utc,
       (outside_air_temp_f - 32) * 5 / 9.0 AS outside_air_temp_c
FROM ahu_1
UNION ALL
SELECT 'AHU_2', timestamp_utc, (outside_air_temp_f - 32) * 5 / 9.0
FROM ahu_2
"""
conn.execute(STACK_VIEW_SQL)

# 3. What the stacked table holds, per AHU
STACK_SUMMARY_SQL = """
SELECT equipment_id, count(*) AS rows,
       min(timestamp_utc) AS first_ts, max(timestamp_utc) AS last_ts,
       avg(outside_air_temp_c) AS mean_oa_c, min(outside_air_temp_c) AS
         min_oa_c
FROM stacked
GROUP BY equipment_id
ORDER BY equipment_id
"""
```

```
# 4. Join on timestamp: one row per timestamp, one column per AHU. For
        each ahu_1 row SQLite SEEKS
#    the matching
#    ahu_2 row through the primary key's B-tree (an index nested-loop
        join): O(n log m).
JOIN_SQL = """
FROM ahu_1 a
JOIN ahu_2 b ON b.timestamp_utc = a.timestamp_utc
"""
PAIRS_SQL = f"""
CREATE TEMP VIEW pairs AS
SELECT a.timestamp_utc,
       (a.outside_air_temp_f - 32) * 5 / 9.0 AS oa_ahu_1_c,
       (b.outside_air_temp_f - 32) * 5 / 9.0 AS oa_ahu_2_c,
       (a.outside_air_temp_f - 32) * 5 / 9.0 - (b.outside_air_temp_f -
         32) * 5 / 9.0 AS diff_k
{JOIN_SQL}
"""
conn.execute(PAIRS_SQL)

# 5. How far apart are they?
GAP_SQL = """
SELECT count(*) AS paired_rows,
       avg(abs(diff_k)) AS mean_abs_diff_k,
       max(abs(diff_k)) AS max_abs_diff_k,
       sum(abs(diff_k) > 2)  AS rows_over_2k,
       sum(abs(diff_k) > 20) AS rows_over_20k
FROM pairs
"""

# 6. Join explosion: the first day (280 rows per AHU), joined on the
        HOUR instead of the timestamp.
#    The hour repeats 12 times per AHU, so every row matches 12 rows on
        the other side.
#    SQL has no validate="1:1"; the equivalent is checking that the
        join key is unique.
EXPLOSION_SQL = """
WITH da AS (SELECT timestamp_utc, substr(timestamp_utc, 1, 13) AS hour
         FROM ahu_1
            WHERE timestamp_utc < :day_end),
     db AS (SELECT timestamp_utc, substr(timestamp_utc, 1, 13) AS hour
         FROM ahu_2
            WHERE timestamp_utc < :day_end)
SELECT (SELECT count(*) FROM da) AS rows_per_side,
       (SELECT count(*) FROM da JOIN db ON db.timestamp_utc =
         da.timestamp_utc)
         AS joined_on_timestamp,
       (SELECT count(*) FROM da JOIN db ON db.hour = da.hour) AS
         joined_on_hour,
       (SELECT count(*) > count(DISTINCT hour) FROM db) AS
         validate_blocked
"""

stack_summary = pd.read_sql_query(STACK_SUMMARY_SQL, conn)
pairs = pd.read_sql_query("SELECT * FROM pairs ORDER BY timestamp_utc",
         conn)
gap = pd.read_sql_query(GAP_SQL, conn)
explosion = pd.read_sql_query(EXPLOSION_SQL, conn, params=PARAMS)
```

```
RESULTS = {
    "stack_summary": stack_summary,
    "oa_pairs": pairs,
    "oa_gap": gap,
    "explosion": explosion,
}

print("Stacked table, per AHU:")
print(stack_summary.round(2).to_string(index=False))
print("\nOutside-air temperature, AHU_1 minus AHU_2 (K):")
print(gap.round(2).to_string(index=False))
print("\nJoin explosion on one day of data:")
print(explosion.to_string(index=False))

# The plans and a timing (not part of RESULTS): how SQLite runs the
        join.
conn.execute("CREATE TEMP TABLE a_plain AS SELECT timestamp_utc,
        outside_air_temp_f FROM ahu_1")
conn.execute("CREATE TEMP TABLE b_plain AS SELECT timestamp_utc,
        outside_air_temp_f FROM ahu_2")
COUNT_PLAIN = "SELECT count(*) FROM a_plain a JOIN b_plain b ON
        b.timestamp_utc = a.timestamp_utc"
COUNT_PK = "SELECT count(*) FROM ahu_1 a JOIN ahu_2 b ON
        b.timestamp_utc = a.timestamp_utc"
print("\nQuery plan, join on the primary key:")
for row in conn.execute("EXPLAIN QUERY PLAN " + COUNT_PK):
    print("  ", row[3])
print("Query plan, join on tables with no index (SQLite builds a
        temporary one):")
for row in conn.execute("EXPLAIN QUERY PLAN " + COUNT_PLAIN):
    print("  ", row[3])
t0 = time.perf_counter()
conn.execute(COUNT_PK).fetchone()
t_pk = time.perf_counter() - t0
t0 = time.perf_counter()
conn.execute(COUNT_PLAIN).fetchone()
t_auto = time.perf_counter() - t0
print(f"\njoin via primary-key seeks:          {t_pk * 1000:8.1f} ms")
print(f"join via temporary automatic index:  {t_auto * 1000:8.1f} ms")
```

Day 8 — CASE and conditional aggregation: pressure while running, hours while unoccupied

Goal: ask conditional questions ("static pressure *while the fan runs*", "run hours *while unoccupied*") with `CASE` in SQL and with masks, `np.where`, `np.select` and `groupby` in pandas, and see why vectorized beats row-by-row.

Read

A fan that is off has no duct static pressure. If you average `da_p_inwc` over all rows you mix "the fan was off" into "the pressure was low", and the number means nothing. Almost every fault rule starts by **gating**: judge only the rows where the equipment is doing its job. That needs a condition inside the query.

`CASE` is SQL's if/else for one row: `CASE WHEN fan = 1 THEN 'running' ELSE 'off' END`. The first `WHEN` that matches wins; `ELSE` is the default (leave it out and you get `NULL`). Use it two ways:

1. **As a column or `GROUP BY` key.** `GROUP BY CASE WHEN supply_fan_status = 1 ...` gives one group per state.
2. **Inside an aggregate: conditional aggregation.** In SQLite a comparison evaluates to 0 or 1, so `sum(supply_fan_status = 1 AND occ_c <> 1)` *counts the rows* where both hold (other databases spell it `sum(CASE WHEN ... THEN 1 ELSE 0 END)`). Many different counts come out of one scan: total rows, run rows, rows that run while unoccupied, rows occupied.

The pandas pieces:

Idea	`day08_sql.py`	`day08_pandas.py`
a condition	`supply_fan_status = 1`	`running = h.supply_fan_status == 1` (a boolean *mask*)
combine	`AND`, `OR`, `NOT`	`&`, `|`, `~` (parenthesize each side)
two outcomes	`CASE WHEN c THEN a ELSE b END`	`np.where(c, a, b)`
many outcomes	`CASE WHEN c1 THEN a WHEN c2 THEN b ELSE d END`	`np.select([c1, c2], [a, b], default=d)`
per-group counts	`GROUP BY equipment_id, sum(cond)`	`h.assign(flag=cond).groupby("equipment_id")["flag"].sum()`

Idea	day08_sql.py	day08_pandas.py
fraction of rows	`100.0 * sum(occ_c = 1) / count(*)`	`(h.occ_c == 1).mean() * 100`

Careful with `<>` and `NULL`: `occ_c <> 1` is `NULL`, not true, when `occ_c` is `NULL`. This data has no NULLs, but a real export will; a `CASE` or a mask is where they quietly drop rows. (pandas masks do the same with `NaN`.)

Vectorized vs row-by-row. `np.select` evaluates the whole column in compiled loops. `df.apply(f, axis=1)` calls a Python function once per row, creating a `Series` for each. Both are O(n), but the constants are very different: on 71,072 rows the vectorized version takes about 2 ms and `apply(axis=1)` about half a second, roughly 200× slower, and the gap grows with the data. In pandas, reach for a mask or `np.select` first. SQL is already "vectorized": the engine does the loop, and you describe the condition.

What Building 50 says (both printouts):

```
Duct static pressure by fan state (Pa):
equipment_id fan_state  rows  mean_static_pa  mean_setpoint_pa
mean_error_pa
       AHU_1       off 16643            35.5             329.9
-294.4
       AHU_1   running 18893           335.6             349.3
-13.7
       AHU_2       off 17541            37.6             197.9
-160.3
       AHU_2   running 17995           349.4             188.2
161.2
```

Averaging only the *running* rows shows it: AHU_1 sits 14 Pa under setpoint, AHU_2 about 161 Pa *over* it (Day 14 turns that into a rule). With the fan off the gap is meaningless, so a rule gates it out.

Run

```
uv run python lessons/course.py run 8
```

That runs `day08_pandas.py`, then `day08_sql.py`. The second table:

```
Run hours, occupancy:
equipment_id  rows   run_h  run_h_unoccupied  pct_occupied
pct_run_while_unoccupied
       AHU_1 35536 1574.42             58.58         51.26
3.72
       AHU_2 35536 1499.58             11.17         51.30
0.74
```

`occ_c` is the BAS's occupancy command (1 occupied, 0 unoccupied). Both AHUs are scheduled occupied about 51% of the time and run about 1,500 hours. AHU_1 runs 58.6 hours while unoccupied, 3.7% of its run time; AHU_2 only 11.2 h. Some unoccupied running is normal (morning warm-up, night purge) and some is a schedule override someone forgot about. That ratio is exactly what an "AHU running while unoccupied" rule would look at.

After the tables the pandas script times mask vs `apply`; the SQL script times one conditional-aggregation query against four separate `COUNT` queries (one scan versus four).

Try it

1. Add a count to both scripts: rows where the fan is running but static pressure is below 50 Pa ("running with no pressure"). How many per AHU? How many of those are occupied?
2. Replace `np.select` with two nested `np.where` calls, and the `CASE` with a nested `CASE` (one `CASE` inside the `ELSE` of another). Which reads better with four states?

Answers

1. SQL: `sum(supply_fan_status = 1 AND duct_static_pa < 50)`; pandas: `(running & (h.duct_static_pa < 50))` summed per AHU. AHU_1: 460 rows, 209 of them occupied. AHU_2: 26 rows, 14 occupied. AHU_1's fan "runs" with no pressure far more often: a failed fan or a status point that lies. The difference is the kind of thing Day 14's FC1 rule flags.
2. `np.where(a, "x", np.where(b, "y", np.where(c, "z", "w")))` works, but each level hides one more condition; four states is where `np.select` (flat list of condition → label) and a flat `CASE` pull ahead.

Why it matters

"While running", "while occupied", "while unoccupied" are the words every fault rule starts with. In Day 14 they become the gate `supply_fan_status = 1`, and in later weeks they come from the data model instead of a hard-coded column. Conditional aggregation is also how you build a dashboard row (hours run, hours faulted, percent occupied) in one pass over the data. Day 9 groups the same rows by time instead of by condition.

day08_pandas.py

```python
"""Day 8, pandas: conditions. Static pressure while running vs off, run
        hours while unoccupied.

    uv run python lessons/unit1-sqlite/day08_pandas.py

SQL's CASE becomes boolean masks, np.where (two outcomes) and np.select
        (many). Conditional
aggregation (`sum(CASE WHEN ... THEN 1 END)`) becomes "sum a mask", or
        "filter, then groupby".
Compare the printout with day08_sql.py: the numbers should match.
"""

import time
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

PA_PER_INH2O = 249.08891  # Day 6: the historian logged inches of
        water; fault math is SI

# 1. Load + stack both AHUs
COLUMNS = ["timestamp_utc", "supply_fan_status", "da_p_inwc",
        "da_p_setpoint_inwc", "occ_c"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip / "history_wide.csv",
        usecols=COLUMNS)
    df.insert(0, "equipment_id", equip)
    frames.append(df)
h = pd.concat(frames, ignore_index=True)

# 2. SI units
h["duct_static_pa"] = h["da_p_inwc"] * PA_PER_INH2O
h["duct_static_sp_pa"] = h["da_p_setpoint_inwc"] * PA_PER_INH2O

# 3. Masks: one boolean per condition. np.where picks one of two
        values, np.select the first
#    matching of many.
running = h["supply_fan_status"] == 1
occupied = h["occ_c"] == 1
h["fan_state"] = np.where(running, "running", "off")
h["state"] = np.select(
    [running & occupied, running & ~occupied, ~running & occupied],
    ["run_occ", "run_unocc", "off_occ"],
    default="off_unocc",
)
states = h[["equipment_id", "timestamp_utc", "state"]]

# 4. Static pressure while running vs off (a CASE as the GROUP BY key)
h["static_error_pa"] = h["duct_static_pa"] - h["duct_static_sp_pa"]
static_by_state = (
    h.groupby(["equipment_id", "fan_state"])
```

```
    .agg(
        rows=("timestamp_utc", "size"),
        mean_static_pa=("duct_static_pa", "mean"),
        mean_setpoint_pa=("duct_static_sp_pa", "mean"),
        mean_error_pa=("static_error_pa", "mean"),
    )
    .reset_index()
)

# 5. Conditional aggregation: several different counts in one pass, one
        row per AHU
by_ahu = h.assign(run_unocc=running & ~occupied, run=running,
        occ=occupied).groupby("equipment_id")
occupancy = pd.DataFrame(
    {
        "rows": by_ahu.size(),
        "run_h": by_ahu["run"].sum() * 5 / 60,
        "run_h_unoccupied": by_ahu["run_unocc"].sum() * 5 / 60,
        "pct_occupied": by_ahu["occ"].mean() * 100,
    }
).reset_index()
occupancy["pct_run_while_unoccupied"] = 100 *
        occupancy["run_h_unoccupied"] / occupancy["run_h"]

RESULTS = {"states": states, "static_by_state": static_by_state,
        "occupancy": occupancy}

print("Duct static pressure by fan state (Pa):")
print(static_by_state.round(1).to_string(index=False))
print("\nRun hours, occupancy:")
print(occupancy.round(2).to_string(index=False))
print("\nRows per state:")
print(states.groupby(["equipment_id",
        "state"]).size().unstack().to_string())

# Timing (not part of RESULTS): the same classification as a vectorized
        mask and as a row-by-row
#    apply.
sample = h[["supply_fan_status", "occ_c"]]


def classify(row):
    if row["supply_fan_status"] == 1:
        return "run_occ" if row["occ_c"] == 1 else "run_unocc"
    return "off_occ" if row["occ_c"] == 1 else "off_unocc"


t0 = time.perf_counter()
vec = np.select(
    [running & occupied, running & ~occupied, ~running & occupied],
    ["run_occ", "run_unocc", "off_occ"],
    default="off_unocc",
)
t_vec = time.perf_counter() - t0
t0 = time.perf_counter()
rowwise = sample.apply(classify, axis=1)
t_apply = time.perf_counter() - t0
```

```
print(f"\nvectorized np.select, {len(h):,} rows: {t_vec * 1000:8.1f}
        ms")
print(
    f"apply(axis=1), same rows:      {t_apply * 1000:8.1f} ms  "
    f"(same answer: {(rowwise == vec).all()})"
)
```

day08_sql.py

```
"""Day 8, SQL: conditions. Static pressure while running vs off, run
        hours while unoccupied.

    uv run python lessons/course.py setup         # once: loads the
        CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day08_sql.py

The same computation as day08_pandas.py, done by SQLite on the same
        rows. CASE picks a value per
row; CASE inside an aggregate (`sum(CASE WHEN ... THEN 1 ELSE 0 END)`)
        counts several things in
one pass.
Compare the printout with day08_pandas.py: the numbers should match.
"""

import os
import sqlite3
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
    "pa_per_inh2o": 249.08891
}  # Day 6: the historian logged inches of water; fault math is SI

# 1 + 2. Stack both AHUs and convert to SI. ahu_1 and ahu_2 are the
        CSVs loaded as tables.
#        (CREATE VIEW can't take bound parameters, so the constant is
        spliced into the text.)
H_VIEW_SQL = """
CREATE TEMP VIEW h AS
SELECT 'AHU_1' AS equipment_id, timestamp_utc, supply_fan_status,
        occ_c,
       da_p_inwc * {pa} AS duct_static_pa, da_p_setpoint_inwc * {pa} AS
        duct_static_sp_pa
FROM ahu_1
UNION ALL
SELECT 'AHU_2', timestamp_utc, supply_fan_status, occ_c,
       da_p_inwc * {pa}, da_p_setpoint_inwc * {pa}
FROM ahu_2
"""
conn.execute(H_VIEW_SQL.format(pa=repr(PARAMS["pa_per_inh2o"])))

# 3. CASE per row: the first WHEN that matches wins (np.select), ELSE
        is the default
STATES_SQL = """
SELECT equipment_id, timestamp_utc,
```

```
       CASE WHEN supply_fan_status = 1 AND occ_c = 1 THEN 'run_occ'
            WHEN supply_fan_status = 1 AND occ_c <> 1 THEN 'run_unocc'
            WHEN supply_fan_status <> 1 AND occ_c = 1 THEN 'off_occ'
            ELSE 'off_unocc' END AS state
FROM h
ORDER BY equipment_id, timestamp_utc
"""

# 4. Static pressure while running vs off: a CASE as the GROUP BY key
STATIC_SQL = """
SELECT equipment_id,
       CASE WHEN supply_fan_status = 1 THEN 'running' ELSE 'off' END AS
         fan_state,
       count(*) AS rows,
       avg(duct_static_pa)                     AS mean_static_pa,
       avg(duct_static_sp_pa)                  AS mean_setpoint_pa,
       avg(duct_static_pa - duct_static_sp_pa)  AS mean_error_pa
FROM h
GROUP BY equipment_id, fan_state
ORDER BY equipment_id, fan_state
"""

# 5. Conditional aggregation: several different counts in one pass, one
         row per AHU.
#    A comparison is 0 or 1 in SQLite, so sum(<condition>) counts the
         rows where it holds.
OCCUPANCY_SQL = """
SELECT equipment_id,
       count(*) AS rows,
       sum(supply_fan_status = 1) * 5 / 60.0                       AS
         run_h,
       sum(supply_fan_status = 1 AND occ_c <> 1) * 5 / 60.0        AS
         run_h_unoccupied,
       100.0 * sum(occ_c = 1) / count(*)                           AS
         pct_occupied,
       100.0 * sum(supply_fan_status = 1 AND occ_c <> 1) /
         sum(supply_fan_status = 1)
                                                                   AS
         pct_run_while_unoccupied
FROM h
GROUP BY equipment_id
ORDER BY equipment_id
"""

states = pd.read_sql_query(STATES_SQL, conn)
static_by_state = pd.read_sql_query(STATIC_SQL, conn)
occupancy = pd.read_sql_query(OCCUPANCY_SQL, conn)

RESULTS = {"states": states, "static_by_state": static_by_state,
         "occupancy": occupancy}

print("Duct static pressure by fan state (Pa):")
print(static_by_state.round(1).to_string(index=False))
print("\nRun hours, occupancy:")
print(occupancy.round(2).to_string(index=False))
print("\nRows per state:")
print(states.groupby(["equipment_id",
         "state"]).size().unstack().to_string())
```

```
# Timing (not part of RESULTS): conditional aggregation = one scan, vs
        one query (one scan) per
#    number.
t0 = time.perf_counter()
conn.execute(OCCUPANCY_SQL).fetchall()
t_one = time.perf_counter() - t0
SEPARATE = [
    "SELECT count(*) FROM h",
    "SELECT count(*) FROM h WHERE supply_fan_status = 1",
    "SELECT count(*) FROM h WHERE supply_fan_status = 1 AND occ_c <>
        1",
    "SELECT count(*) FROM h WHERE occ_c = 1",
]
t0 = time.perf_counter()
for q in SEPARATE:
    conn.execute(q).fetchall()
t_four = time.perf_counter() - t0
print(f"\none query, conditional aggregation: {t_one * 1000:7.1f} ms")
print(f"four separate COUNT queries:        {t_four * 1000:7.1f} ms")
```

Day 9 — Time buckets: hourly and daily averages, and views vs summary tables

Goal: collapse 5-minute rows into hourly and daily summaries in SQL and pandas (what Timescale calls `time_bucket`), and understand when to store a query (a view) and when to store its result (a summary table).

Read

A year of 5-minute data is 105,000 rows per point. A dashboard, a report or a daily-energy number wants hours or days. **Bucketing** means: give every row a key for "which hour (or day) am I in", then `GROUP BY` that key. Only the key changes; everything else is Day 8's conditional aggregation.

Bucket	`day09_sql.py`	`day09_pandas.py`
hour, as text	`strftime('%Y-%m-%d %H', timestamp_utc)`	`ts.dt.strftime("%Y-%m-%d %H")`
hour, as an integer	`unixepoch(timestamp_utc) / 3600`	`(ts - epoch).dt.total_seconds() // 3600`
hour, as a timestamp	integer bucket `* 3600`, back through `strftime(..., 'unixepoch')`	`ts.dt.floor("h")`
day	`strftime('%Y-%m-%d', timestamp_utc)` or `date(timestamp_utc)`	`ts.dt.strftime("%Y-%m-%d")` or `.dt.floor("D")`
group + average	`GROUP BY equipment_id, bucket` + `avg(...)`	`groupby(["equipment_id", bucket]).agg(...)`
the same, as one call		`df.set_index("ts").groupby(eq)[col].resample("h").mean()`

Two things to know:

- **The integer bucket is the portable, fast one.** `unixepoch(ts) / 3600` is integer division (SQLite divides integer by integer exactly like this), so the key is a number: cheap to compare, easy to index. `strftime` gives readable text. Both find the same buckets, which the scripts' "bucket check" table confirms (2,962 per AHU).
- **`resample` fills empty buckets; `groupby` doesn't.** With `resample("h")` an hour with no data appears as a row of `NaN`; with `groupby`/`GROUP BY` it simply isn't there. Building 50 has no gaps, so both agree (5,924 hourly rows), but a real building will have some, and you have to choose which behavior you want.

The columns summarize like Day 8: `avg(outside_air_temp_c)` over all rows, `avg(static_running_pa)` over the rows where the fan was running (the column is `NULL` while the fan is off, and `avg()` skips `NULL`, like pandas' `mean()` skips `NaN`), and `avg(supply_fan_status)`, the fraction of the hour the fan was on. A bucket with no running rows has no mean: `NaN` in pandas, `NULL` in SQL, which the parity test treats as equal.

View vs summary table. Once you have an hourly query, there are two ways to keep it:

- `CREATE VIEW hourly_v AS SELECT ...` stores the **query**. Every read re-runs the aggregation over all 71,072 rows. It can't go stale, and costs no storage.
- `CREATE TABLE hourly_t AS SELECT ...` stores the **result**: 5,924 rows, computed once. Reads are instant (and can be indexed), but the table is a snapshot: new rows in `ahu_1` don't show up until you rebuild it.

This is the same trade-off as caching anywhere: compute-on-read versus compute-once. It's also what Timescale calls a *continuous aggregate*: a summary table the database keeps fresh for you. The scripts read the summary ten times each way: about a second for the view, a few milliseconds for the table (pandas shows the same shape: re-running `groupby` every time vs reusing the DataFrame). The right choice depends on how often the data changes versus how often it is read; dashboards that read all day over history that rarely changes want a table.

Run

```
uv run python lessons/course.py run 9
```

That runs `day09_pandas.py`, then `day09_sql.py`. Both print the first rows of the hourly and daily tables, and one day of hourly outside-air temperature. An excerpt:

```
Hourly averages, first rows (AHU_1):
equipment_id                hour_start  rows  mean_oa_c
mean_static_running_pa  fan_on_fraction
       AHU_1 2026-03-16 00:00:00+00:00     4      20.57
NaN             0.0
       AHU_1 2026-03-16 01:00:00+00:00    12      19.98
NaN             0.0
...
hourly rows: 5,924, daily rows: 248
```

(SQL shows a `NULL` as `NaN` too, since pandas prints it.) The first bucket has only **4 rows** because the record starts at 00:40: a partial hour. A rule that needs "12 samples per hour" has to know about partial buckets at the edges.

Try it

1. Which day has the highest mean outside-air temperature on AHU_1, and which the lowest? Change `daily` to find out (hint: sort, or `idxmax` / `ORDER BY ... DESC LIMIT 1`).
2. Change the hourly buckets to 15-minute buckets in both scripts. What are the new key expressions, and how many rows does AHU_1 get?

Answers

1. Hottest: 2026-07-01 at 31.3 °C. Coldest: 2026-03-17 at −5.9 °C (the record opens with a flat, stuck-looking reading of 20.6 °C, a reminder to look at the data before trusting a mean). SQL: `ORDER BY mean_oa_c DESC LIMIT 1`; pandas: `daily.loc[daily.mean_oa_c.idxmax()]`.
2. SQL: `unixepoch(timestamp_utc) / 900` (900 s), turned back with `* 900`. pandas: `ts.dt.floor("15min")`. The data is 5-minute steps, so each bucket has three rows and AHU_1 gets 35,536 / 3 ≈ 11,850 buckets (about 11,846).

Why it matters

Hourly and daily summaries are what people read, and what slower rules (daily energy, hours outside setpoint) run on. Bucketing is also where timezones bite: these buckets are UTC, so a “day” starts in the evening in US time zones. A report for building staff has to bucket in local time. And the view/table choice is the first taste of a decision you’ll make repeatedly: derive on read, or materialize once. Day 10 does a different kind of time join: matching each row to the fault episode it falls in.

day09_pandas.py

```
"""Day 9, pandas: time buckets. Hourly and daily averages, recompute vs
        compute once.

    uv run python lessons/unit1-sqlite/day09_pandas.py

A bucket is "which hour (day) does this 5-minute row belong to". pandas
        has three ways to say it:
`dt.floor` + `groupby`, `resample`, or plain integer division. SQL's
        `strftime` and
`unixepoch()/3600` map to them.
Compare the printout with day09_sql.py: the numbers should match.
"""

import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

PA_PER_INH2O = 249.08891  # Day 6: the historian logged inches of
        water; fault math is SI
EPOCH = pd.Timestamp("1970-01-01", tz="UTC")

# 1. Load + stack both AHUs
COLUMNS = ["timestamp_utc", "outside_air_temp_f", "da_p_inwc",
        "supply_fan_status"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip / "history_wide.csv",
        usecols=COLUMNS)
    df.insert(0, "equipment_id", equip)
    frames.append(df)
h = pd.concat(frames, ignore_index=True)

# 2. SI units, and real timestamps (the CSV holds text; floor/resample
        need datetimes)
h["ts"] = pd.to_datetime(h["timestamp_utc"], utc=True)
h["outside_air_temp_c"] = (h["outside_air_temp_f"] - 32) * 5 / 9
h["duct_static_pa"] = h["da_p_inwc"] * PA_PER_INH2O
h["static_running_pa"] = h["duct_static_pa"].where(
    h["supply_fan_status"] == 1
)  # NaN when the fan is off


# 3. Hourly: floor each timestamp to its hour, then group
def summarize(df: pd.DataFrame, bucket: pd.Series, name: str) ->
        pd.DataFrame:
    return (
        df.groupby(["equipment_id", bucket.rename(name)])
        .agg(
            rows=("ts", "size"),
            mean_oa_c=("outside_air_temp_c", "mean"),
            mean_static_running_pa=(
                "static_running_pa",
```

```
                "mean",
            ),  # mean skips NaN, like avg() skips NULL
            fan_on_fraction=("supply_fan_status", "mean"),
        )
        .reset_index()
    )


hourly = summarize(h, h["ts"].dt.floor("h"), "hour_start")

# 4. Daily: the same, with the date as the bucket
daily = summarize(h, h["ts"].dt.strftime("%Y-%m-%d"), "day")

# 5. The same hourly buckets two other ways: resample, and integer
        division (hours since 1970)
resampled = h.set_index("ts").groupby("equipment_id")
        ["outside_air_temp_c"].resample("h").mean()
hour_int = (h["ts"] - EPOCH).dt.total_seconds() // 3600  # hours since
        1970, like unixepoch() / 3600
bucket_check = pd.DataFrame(
    {
        "hour_buckets_floor": h.groupby("equipment_id")["ts"].apply(
            lambda s: s.dt.floor("h").nunique()
        ),
        "hour_buckets_int":
         hour_int.groupby(h["equipment_id"]).nunique(),
    }
).reset_index()

# 6. One day of the hourly table: what a dashboard asks for
july1 = hourly[
    (hourly.equipment_id == "AHU_1") &
        (hourly.hour_start.dt.strftime("%Y-%m-%d") == "2026-07-01")
]

RESULTS = {"hourly": hourly, "daily": daily, "bucket_check":
        bucket_check, "ahu_1_july_1": july1}

print("Hourly averages, first rows (AHU_1):")
print(hourly[hourly.equipment_id ==
        "AHU_1"].head(3).round(2).to_string(index=False))
print("\nDaily averages, first rows (AHU_1):")
print(daily[daily.equipment_id ==
        "AHU_1"].head(3).round(2).to_string(index=False))
print(f"\nhourly rows: {len(hourly):,}, daily rows: {len(daily):,}")
print("\nSame hourly buckets by floor and by integer division:")
print(bucket_check.to_string(index=False))
same = resampled.dropna().to_numpy()
print(f"hourly means: {hourly['mean_oa_c'].notna().sum():,}  (resample
        gives {len(same):,})")
print("\nAHU_1, 2026-07-01, mean outside air by hour (C):")
print(july1.set_index(july1["hour_start"].dt.strftime("%H:%M"))
        ["mean_oa_c"].round(1).to_string())

# Timing (not part of RESULTS): view = compute on every read; summary
        table = compute once.
READS = 10
```

```
def hourly_view() -> pd.DataFrame:  # "a view": rebuild the summary
        every time someone asks
    return summarize(h, h["ts"].dt.floor("h"), "hour_start")


t0 = time.perf_counter()
for _ in range(READS):
    hourly_view().query("equipment_id == 'AHU_1'")
        ["mean_oa_c"].agg(["mean", "max"])
t_view = time.perf_counter() - t0
t0 = time.perf_counter()
for _ in range(READS):
    hourly.query("equipment_id == 'AHU_1'")["mean_oa_c"].agg(["mean",
        "max"])
t_table = time.perf_counter() - t0
print(f"\n{READS} reads, recompute each time:   {t_view * 1000:8.1f}
        ms")
print(f"{READS} reads, compute once and keep: {t_table * 1000:8.1f}
        ms")
```

day09_sql.py

```
"""Day 9, SQL: time buckets. Hourly and daily averages, and a view vs a
        summary table.

    uv run python lessons/course.py setup         # once: loads the
        CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day09_sql.py

The same computation as day09_pandas.py, done by SQLite on the same
        rows. A bucket is "which hour
(day) does this 5-minute row belong to": `unixepoch(ts) / 3600`
        (integer division) or `strftime`
(text) say it in SQL.
Compare the printout with day09_pandas.py: the numbers should match.
"""

import os
import sqlite3
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
    "pa_per_inh2o": 249.08891
}  # Day 6: the historian logged inches of water; fault math is SI

# 1 + 2. Stack both AHUs and convert to SI. ahu_1 and ahu_2 are the
        CSVs loaded as tables.
#        static_running_pa is NULL while the fan is off; avg() skips
        NULLs.
#        (CREATE VIEW can't take bound parameters, so the constant is
        spliced into the text.)
H_VIEW_SQL = """
CREATE TEMP VIEW h AS
SELECT 'AHU_1' AS equipment_id, timestamp_utc, supply_fan_status,
       (outside_air_temp_f - 32) * 5 / 9.0 AS outside_air_temp_c,
       CASE WHEN supply_fan_status = 1 THEN da_p_inwc * {pa} END AS
         static_running_pa
FROM ahu_1
UNION ALL
SELECT 'AHU_2', timestamp_utc, supply_fan_status,
       (outside_air_temp_f - 32) * 5 / 9.0,
       CASE WHEN supply_fan_status = 1 THEN da_p_inwc * {pa} END
FROM ahu_2
"""
conn.execute(H_VIEW_SQL.format(pa=repr(PARAMS["pa_per_inh2o"])))
```

```
# 3. Hourly: integer division. unixepoch() is seconds since 1970; /
        3600 (integers) is "hours since
#    1970".
#    The bucket is turned back into text, in the same format as
        timestamp_utc, for display.
HOURLY_SELECT = """
SELECT equipment_id,
       strftime('%Y-%m-%d %H:%M:%S+00:00', unixepoch(timestamp_utc) /
         3600 * 3600, 'unixepoch')
         AS hour_start,
       count(*) AS rows,
       avg(outside_air_temp_c)  AS mean_oa_c,
       avg(static_running_pa)   AS mean_static_running_pa,
       avg(supply_fan_status)   AS fan_on_fraction
FROM h
GROUP BY equipment_id, unixepoch(timestamp_utc) / 3600
"""

# 4. Daily: the same, with strftime (text) as the bucket
DAILY_SQL = """
SELECT equipment_id, strftime('%Y-%m-%d', timestamp_utc) AS day,
       count(*) AS rows,
       avg(outside_air_temp_c)  AS mean_oa_c,
       avg(static_running_pa)   AS mean_static_running_pa,
       avg(supply_fan_status)   AS fan_on_fraction
FROM h
GROUP BY equipment_id, strftime('%Y-%m-%d', timestamp_utc)
ORDER BY equipment_id, day
"""

# 5. The same hourly buckets by text and by integer division: they must
        find the same number of
#    buckets
BUCKET_CHECK_SQL = """
SELECT equipment_id,
       count(DISTINCT strftime('%Y-%m-%d %H', timestamp_utc)) AS
         hour_buckets_floor,
       count(DISTINCT unixepoch(timestamp_utc) / 3600)        AS
         hour_buckets_int
FROM h
GROUP BY equipment_id
ORDER BY equipment_id
"""

# 6. A view stores the query (every read recomputes); a summary table
        stores the rows (computed
#    once).
conn.execute("CREATE TEMP VIEW hourly_v AS " + HOURLY_SELECT)
conn.execute("CREATE TEMP TABLE hourly_t AS " + HOURLY_SELECT)
conn.execute("CREATE INDEX temp.hourly_t_eq_hour ON hourly_t
        (equipment_id, hour_start)")
JULY1_SQL = """
SELECT * FROM hourly_t
WHERE equipment_id = 'AHU_1' AND hour_start >= '2026-07-01' AND
        hour_start < '2026-07-02'
ORDER BY hour_start
"""
```

```
hourly = pd.read_sql_query("SELECT * FROM hourly_t ORDER BY
         equipment_id, hour_start", conn)
daily = pd.read_sql_query(DAILY_SQL, conn)
bucket_check = pd.read_sql_query(BUCKET_CHECK_SQL, conn)
july1 = pd.read_sql_query(JULY1_SQL, conn)

RESULTS = {"hourly": hourly, "daily": daily, "bucket_check":
         bucket_check, "ahu_1_july_1": july1}

print("Hourly averages, first rows (AHU_1):")
print(hourly[hourly.equipment_id ==
         "AHU_1"].head(3).round(2).to_string(index=False))
print("\nDaily averages, first rows (AHU_1):")
print(daily[daily.equipment_id ==
         "AHU_1"].head(3).round(2).to_string(index=False))
print(f"\nhourly rows: {len(hourly):,}, daily rows: {len(daily):,}")
print("\nSame hourly buckets by text and by integer division:")
print(bucket_check.to_string(index=False))
n_hourly = conn.execute("SELECT count(mean_oa_c) FROM
         hourly_t").fetchone()[0]
print(f"hourly means: {n_hourly:,}")
print("\nAHU_1, 2026-07-01, mean outside air by hour (C):")
print(july1.set_index(july1["hour_start"].str[11:16])
         ["mean_oa_c"].round(1).to_string())

# Timing (not part of RESULTS): view = compute on every read; summary
         table = compute once.
READS = 10
READ_VIEW = "SELECT avg(mean_oa_c), max(mean_oa_c) FROM hourly_v WHERE
         equipment_id = 'AHU_1'"
READ_TABLE = "SELECT avg(mean_oa_c), max(mean_oa_c) FROM hourly_t WHERE
         equipment_id = 'AHU_1'"
t0 = time.perf_counter()
for _ in range(READS):
    conn.execute(READ_VIEW).fetchall()
t_view = time.perf_counter() - t0
t0 = time.perf_counter()
for _ in range(READS):
    conn.execute(READ_TABLE).fetchall()
t_table = time.perf_counter() - t0
print(f"\n{READS} reads, recompute each time:   {t_view * 1000:8.1f} ms
         (view)")
print(f"{READS} reads, compute once and keep: {t_table * 1000:8.1f} ms
         (summary table)")
```

Day 10 — Joining Open-FDD's results: episodes per rule, then tag every row with its fault episode

Goal: count Open-FDD's fault episodes and hours per rule, then attach each 5-minute history row to the episode(s) it falls inside: an *interval join*, in SQL (`BETWEEN` join) and pandas (`searchsorted`, `merge_asof`). Then finish Week 2 with a challenge that uses everything.

Read

Open-FDD writes its findings as **episodes**: one row per stretch of time a rule was in fault, with a start, an end and the rule's name. They ship next to the CSVs in `data/BUILDING_50/*/fdd_events.csv`, and the SQL side has them as the `fdd_events` table (columns `start_time_utc`, `end_time_utc`, `hvac_system`, `rule_name`, `duration_min`, `sample_count`). `hvac_system` is `bld_50_AHU_1`; the scripts strip the prefix to get the same `equipment_id` the history uses. Counting is a plain `GROUP BY`:

```
equipment_id           rule_name  episodes  hours
       AHU_1                 fc1        80  484.1
       AHU_1                fc13        90  510.0
       AHU_2                fc13        77  209.9
```

That answers "how many, how long", but not "what was the AHU doing during them?" For that, the two tables need to meet: the history has one row per instant, the episodes one row per *interval*. An **interval join** matches a row to an episode when `start <= timestamp <= end`.

Step	`day10_sql.py`	`day10_pandas.py`
episodes per rule	`GROUP BY equipment_id, rule_name`	`groupby(["equipment_id", "rule_name"]).agg(...)`
row inside episode	`ON r.timestamp_utc BETWEEN e.start_time_utc AND e.end_time_utc`	`start <= t <= end` on epoch seconds
how many episodes at once	`LEFT JOIN` + `GROUP BY row` + `count(e.start_time_utc)`	`searchsorted(starts, t, "right") - searchsorted(ends, t, "left")`
latest episode that started	`SELECT max(start) ... WHERE start <= row_ts` (index seek)	`pd.merge_asof(rows, episodes, by="equipment_id")`

Why `searchsorted` works. Sort the starts and sort the ends, once. For a time `t`, the episodes that have started are `#(start <= t)`, the ones that already finished are `#(end < t)`, and the difference is how many contain `t`. Each count is a binary search, so tagging n rows against E episodes costs O((n + E) log E), where the obvious every-row-against-every-episode comparison is O(n · E). On the 35,536 × 90 rows here it's 1.7 ms against 10 ms with numpy broadcasting; against a pure-Python double loop the gap is orders of magnitude.

Why SQL needs an index here. The `BETWEEN` join is a "range join": `ev` has an index on `(equipment_id, start_time_utc)`, so for each row SQLite seeks to the episodes with `start_time_utc <= row time` (the plan says `SEARCH e USING INDEX ev_eq_start (equipment_id=? AND start_time_utc<?)`) and then filters on the end. A B-tree can only narrow one end of an interval, which is why interval joins are slower than equality joins (about a second here for 71,072 rows, against tens of milliseconds for Day 7's join).

As-of vs interval. `merge_asof` is built for "the most recent thing before this row" (a price, a setpoint, a status change), and it's fast because both sides are sorted. It returns **one** match, the latest start at or before the row; the script keeps it only if that episode hasn't ended yet. That is correct *if episodes don't overlap*. Open-FDD's episodes **do** overlap heavily: AHU_1's `fc13` has 90 episodes and, at the busiest moment, 15 open at once (`max_open`). That also explains why the summed `hours` above (510 h) is so much larger than the time the rows are actually covered (1,206 rows × 5 min = 100.5 h): the episodes overlap and the sum counts the overlap many times. Count rows or merge the intervals, don't add up `duration_min` when intervals overlap. (`pd.IntervalIndex` is the other pandas option, but its `get_indexer` raises `InvalidIndexError: cannot handle overlapping indices` for this data; `get_indexer_non_unique` is the overlapping-friendly version.)

Here the as-of lookup misses no rows (`rows_missed_by_asof` is 0), because the overlapping episodes share the same end. That's luck of this data, not a property you can rely on: a short episode starting inside a long one would hide the long one from `merge_asof`. The `episodes_open` count is the one to trust.

Run

```
uv run python lessons/course.py run 10
```

That runs `day10_pandas.py`, then `day10_sql.py`. Both print:

```
Every 5-minute row tagged with its fc13 episodes:
equipment_id  rows  rows_in_episode  max_open  rows_missed_by_asof
covered_h  episode_hours_summed
       AHU_1 35536             1206        15                    0
100.5                510.0
       AHU_2 35536              954        10                    0
79.5                209.9
equipment_id             timestamp_utc  episodes_open
latest_start
       AHU_1 2026-04-09 15:25:00+00:00              1 2026-04-09
15:23:30+00:00
```

A tagged row carries the start of its episode (`latest_start`). Timestamps are ISO text in one format, so SQL compares the strings directly and pandas converts them to seconds since 1970; the results are identical. At the end each script prints a timing: pandas, brute force vs binary search; SQL, the `BETWEEN` join vs the as-of lookup.

Try it

1. Change `RULE` / `rule` to `"fc1"` in both scripts. What do you get for AHU_2, and why is that a trap for a naive `INNER JOIN` version of the query?
2. **Week 2 challenge.** Use the tagged rows. Add `supply_fan_status` and `outside_air_temp_f` to the history load. For each AHU, split the rows into *inside an fc13 episode* and *outside*, and report the row count, the percent of rows with the fan running, and the mean outside-air temperature in °C. What does it say about when fc13 fires?

Answers

1. AHU_2 has no `fc1` episodes at all. With a `LEFT JOIN` every row still appears with `episodes_open = 0`; with an `INNER JOIN` AHU_2 would vanish from the result, and a report built on it would look as though AHU_2 wasn't analyzed rather than healthy. Missing groups are a join's most common silent failure.

2. Result (both languages agree):

   ```
   equipment_id  in_episode   rows  pct_fan_running  mean_oa_c
          AHU_1       False  34330             51.5       15.9
          AHU_1        True   1206            100.0       26.3
          AHU_2       False  34582             49.3       17.2
          AHU_2        True    954            100.0       27.3
   ```

   Every row inside an `fc13` episode has the fan running (100%), and it's warm outside: 26–27 °C against 16–17 °C otherwise. The rule fires when the AHU is working in warm weather, which is when you'd expect a cooling-related rule to find trouble. SQL, with the `EXISTS` standing in for the join:

```
SELECT h.equipment_id,
       EXISTS (SELECT 1 FROM ev WHERE ev.equipment_id =
         h.equipment_id
                AND h.timestamp_utc BETWEEN ev.start_time_utc AND
         ev.end_time_utc) AS in_episode,
       count(*) AS rows, 100.0 * avg(supply_fan_status) AS
         pct_fan_running,
       avg((outside_air_temp_f - 32) * 5 / 9.0) AS mean_oa_c
FROM h GROUP BY 1, 2
```

pandas: merge the history columns onto `tagged`, add `in_episode = tagged.episodes_open > 0` and `oa_c = (outside_air_temp_f - 32) * 5 / 9`, then `groupby(["equipment_id", "in_episode"]).agg(...)`. This uses Day 6 (units), Day 7 (join), Day 8 (conditional aggregation) and today's interval join.

Why it matters

Open-FDD tells you *that* a rule fired; the interval join is how you ask *what the equipment was doing* when it did: the fan state, the weather, the setpoint, the schedule. Day 14 already compared your own rule with Open-FDD's episodes this way, and later weeks do it again. Watch for the two traps from this week that bite here too: a join on a repeated key multiplies rows (Day 7), and summing episode durations double counts where episodes overlap.

day10_pandas.py

```
"""Day 10, pandas: Open-FDD's results. Episodes per rule, then tag each
        row with its episode.

    uv run python lessons/unit1-sqlite/day10_pandas.py

`fdd_events.csv` holds one row per fault *episode* (start, end, rule).
        The interval join asks, for
each 5-minute history row: which episode(s) is this timestamp inside?
        Two pandas answers: binary
search with np.searchsorted (counts all episodes, even overlapping
        ones) and merge_asof (the
latest-starting episode).
Compare the printout with day10_sql.py: the numbers should match.
"""

import time
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"

RULE = "fc13"  # which Open-FDD rule to tag the rows with
EPOCH = pd.Timestamp("1970-01-01", tz="UTC")


def seconds(text: pd.Series) -> pd.Series:
    """ISO text -> seconds since 1970 (a number compares and sorts
        cheaply)."""
    return (pd.to_datetime(text, utc=True) - EPOCH).dt.total_seconds()


# 1. Load: Open-FDD's episodes for both AHUs, and the history
        timestamps
events = pd.concat(
    [pd.read_csv(DATA / e / "fdd_events.csv") for e in ("AHU_1",
        "AHU_2")], ignore_index=True
)
events["equipment_id"] = events["hvac_system"].str.replace("bld_50_",
        "", regex=False)
events["start_sec"] = seconds(events["start_time_utc"])
events["end_sec"] = seconds(events["end_time_utc"])

frames = []
for equip in ("AHU_1", "AHU_2"):
    df = pd.read_csv(DATA / equip / "history_wide.csv", usecols=
        ["timestamp_utc"])
    df.insert(0, "equipment_id", equip)
    frames.append(df)
h = pd.concat(frames, ignore_index=True)
h["ts_sec"] = seconds(h["timestamp_utc"])

# 2. Episodes and hours per rule: GROUP BY two keys
rule_summary = (
```

```
    events.groupby(["equipment_id", "rule_name"])
    .agg(episodes=("rule_name", "size"), duration_min=("duration_min",
        "sum"))
    .assign(hours=lambda d: d["duration_min"] / 60)
    .drop(columns="duration_min")
    .reset_index()
)

# 3. Interval join, version 1: how many episodes of RULE is each row
        inside? (start <= t <= end)
#    Sort the starts and the ends once. #(start <= t) - #(end < t) =
        episodes containing t.
#    Two binary searches per row: O((n + E) log E) overall.
fc = events[events["rule_name"] == RULE]
open_count = pd.Series(0, index=h.index, dtype="int64")
for equip, rows in h.groupby("equipment_id"):
    ev = fc[fc["equipment_id"] == equip]
    starts, ends = np.sort(ev["start_sec"].to_numpy()),
        np.sort(ev["end_sec"].to_numpy())
    t = rows["ts_sec"].to_numpy()
    open_count.loc[rows.index] = np.searchsorted(starts, t,
        side="right") - np.searchsorted(
        ends, t, side="left"
    )

# 4. Interval join, version 2: merge_asof = "the latest episode that
        started at or before this row",
#    then keep it only if it has not ended yet. It gives ONE episode
        per row, so with overlapping
#    episodes it can miss a row that an earlier, longer episode still
        covers.
asof = pd.merge_asof(
    h.sort_values("ts_sec"),
    fc[["equipment_id", "start_sec", "end_sec",
        "start_time_utc"]].sort_values("start_sec"),
    left_on="ts_sec",
    right_on="start_sec",
    by="equipment_id",
)
asof["latest_start"] = asof["start_time_utc"].where(asof["ts_sec"] <=
        asof["end_sec"], None)
tagged = (
    h[["equipment_id", "timestamp_utc"]]
    .assign(episodes_open=open_count)
    .merge(
        asof[["equipment_id", "timestamp_utc", "latest_start"]],
        on=["equipment_id", "timestamp_utc"],
    )
    .sort_values(["equipment_id", "timestamp_utc"], ignore_index=True)
)

# 5. Rows covered vs episode hours. 5-minute rows x 5 / 60 = hours the
        rule was "open"
tagged_summary = (
    tagged.assign(
        covered=tagged["episodes_open"] > 0,
        asof_hit=tagged["latest_start"].notna(),
    )
    .assign(missed_by_asof=lambda d: d["covered"] & ~d["asof_hit"])
```

```
    .groupby("equipment_id")
    .agg(
        rows=("timestamp_utc", "size"),
        rows_in_episode=("covered", "sum"),
        max_open=("episodes_open", "max"),
        rows_missed_by_asof=("missed_by_asof", "sum"),
    )
    .assign(covered_h=lambda d: d["rows_in_episode"] * 5 / 60)
    .reset_index()
)
episode_h = rule_summary[rule_summary["rule_name"] == RULE]
        [["equipment_id", "hours"]]
tagged_summary = tagged_summary.merge(
    episode_h.rename(columns={"hours": "episode_hours_summed"}),
        on="equipment_id"
)

RESULTS = {"rule_summary": rule_summary, "tagged": tagged,
        "tagged_summary": tagged_summary}

print("Open-FDD episodes and hours per rule:")
print(rule_summary.round(1).to_string(index=False))
print(f"\nEvery 5-minute row tagged with its {RULE} episodes:")
print(tagged_summary.round(1).to_string(index=False))
print(tagged[tagged["episodes_open"] >
        0].head(3).to_string(index=False))

# Timing (not part of RESULTS): the same count by brute force (compare
        every row with every
#    episode).
sub = h[h["equipment_id"] == "AHU_1"]
ev = fc[fc["equipment_id"] == "AHU_1"]
t0 = time.perf_counter()
t = sub["ts_sec"].to_numpy()[:, None]
brute = ((ev["start_sec"].to_numpy() <= t) & (t <=
        ev["end_sec"].to_numpy())).sum(axis=1)
t_brute = time.perf_counter() - t0
t0 = time.perf_counter()
tt = sub["ts_sec"].to_numpy()
fast = np.searchsorted(np.sort(ev["start_sec"].to_numpy()), tt,
        "right") - np.searchsorted(
    np.sort(ev["end_sec"].to_numpy()), tt, "left"
)
t_fast = time.perf_counter() - t0
print(f"\nbrute force, {len(sub):,} rows x {len(ev)} episodes: {t_brute
        * 1000:6.1f} ms")
print(f"binary search, same answer ({(brute == fast).all()}):   {t_fast
        * 1000:6.1f} ms")
```

day10_sql.py

```
"""Day 10, SQL: Open-FDD's results. Episodes per rule, then tag every
        5-minute row with its episode.

    uv run python lessons/course.py setup         # once: loads the
        CSVs into var/lessons.sqlite
    uv run python lessons/unit1-sqlite/day10_sql.py

The same computation as day10_pandas.py, done by SQLite on the same
        rows. `fdd_events` holds one
row per fault *episode* (start, end, rule). The interval join asks, for
        each 5-minute history row:
which episode(s) is this timestamp inside? A `BETWEEN` join counts all
        of them; an "as-of" lookup
finds the latest-starting one.
Compare the printout with day10_pandas.py: the numbers should match.
"""

import os
import sqlite3
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {"rule": "fc13"}  # which Open-FDD rule to tag the rows with

# 1. Load: Open-FDD's episodes with an equipment_id, indexed for range
        lookups; history timestamps
#    for both AHUs.
#    The timestamps are ISO text in one format, so comparing the text
        compares them in time order.
conn.execute(
    """
    CREATE TEMP TABLE ev AS
    SELECT replace(hvac_system, 'bld_50_', '') AS equipment_id,
        rule_name,
           start_time_utc, end_time_utc,
           duration_min
    FROM fdd_events
    """
)
conn.execute("CREATE INDEX temp.ev_eq_start ON ev (equipment_id,
        start_time_utc)")
conn.execute(
    """
    CREATE TEMP VIEW h AS
    SELECT 'AHU_1' AS equipment_id, timestamp_utc FROM ahu_1
    UNION ALL
```

```
    SELECT 'AHU_2', timestamp_utc FROM ahu_2
    """
)

# 2. Episodes and hours per rule: GROUP BY two keys
RULE_SUMMARY_SQL = """
SELECT equipment_id, rule_name, count(*) AS episodes, sum(duration_min)
        / 60.0 AS hours
FROM ev
GROUP BY equipment_id, rule_name
ORDER BY equipment_id, rule_name
"""

# 3. Interval join, version 1: a BETWEEN join. Each row pairs with
        every episode that contains it,
#    and
#    GROUP BY row + count() gives the number of episodes open at that
        time. A row inside no episode
#    survives because of the LEFT JOIN (count(e.start_time_utc) skips
        the NULL).
OPEN_COUNT_SQL = """
CREATE TEMP TABLE open_count AS
SELECT r.equipment_id, r.timestamp_utc, count(e.start_time_utc) AS
        episodes_open
FROM h r
LEFT JOIN ev e
  ON e.equipment_id = r.equipment_id
 AND e.rule_name = :rule
 AND r.timestamp_utc BETWEEN e.start_time_utc AND e.end_time_utc
GROUP BY r.equipment_id, r.timestamp_utc
"""
conn.execute(OPEN_COUNT_SQL, PARAMS)

# 4. Interval join, version 2: an "as-of" lookup. Seek the latest
        episode that started at or before
#    the row
#    (an index seek on ev), keep it only if it has not ended yet. One
        episode per row, so with
#    overlapping
#    episodes it can miss a row that an earlier, longer episode still
        covers.
TAGGED_SQL = """
CREATE TEMP TABLE tagged AS
WITH last AS (
  SELECT r.equipment_id, r.timestamp_utc,
         (SELECT max(e.start_time_utc) FROM ev e
          WHERE e.equipment_id = r.equipment_id AND e.rule_name = :rule
            AND e.start_time_utc <= r.timestamp_utc) AS last_start
  FROM h r
)
SELECT l.equipment_id, l.timestamp_utc, o.episodes_open,
       (SELECT CASE WHEN e.end_time_utc >= l.timestamp_utc THEN
         e.start_time_utc END
        FROM ev e
        WHERE e.equipment_id = l.equipment_id AND e.rule_name = :rule
          AND e.start_time_utc = l.last_start) AS latest_start
FROM last l
JOIN open_count o ON o.equipment_id = l.equipment_id AND
        o.timestamp_utc = l.timestamp_utc
```

```
"""
conn.execute(TAGGED_SQL, PARAMS)

# 5. Rows covered vs episode hours. 5-minute rows x 5 / 60 = hours the
        rule was "open"
SUMMARY_SQL = """
SELECT t.equipment_id,
       count(*) AS rows,
       sum(t.episodes_open > 0) AS rows_in_episode,
       max(t.episodes_open) AS max_open,
       sum(t.episodes_open > 0 AND t.latest_start IS NULL) AS
         rows_missed_by_asof,
       sum(t.episodes_open > 0) * 5 / 60.0 AS covered_h,
       (SELECT sum(duration_min) / 60.0 FROM ev e
        WHERE e.equipment_id = t.equipment_id AND e.rule_name = :rule)
         AS episode_hours_summed
FROM tagged t
GROUP BY t.equipment_id
ORDER BY t.equipment_id
"""

rule_summary = pd.read_sql_query(RULE_SUMMARY_SQL, conn)
tagged = pd.read_sql_query("SELECT * FROM tagged ORDER BY equipment_id,
        timestamp_utc", conn)
tagged_summary = pd.read_sql_query(SUMMARY_SQL, conn, params=PARAMS)

RESULTS = {"rule_summary": rule_summary, "tagged": tagged,
        "tagged_summary": tagged_summary}

print("Open-FDD episodes and hours per rule:")
print(rule_summary.round(1).to_string(index=False))
print(f"\nEvery 5-minute row tagged with its {PARAMS['rule']}
        episodes:")
print(tagged_summary.round(1).to_string(index=False))
print(tagged[tagged["episodes_open"] >
        0].head(3).to_string(index=False))

# Plan and timing (not part of RESULTS): the BETWEEN join and the as-of
        lookup both seek into ev's
#    index.
BETWEEN_PLAN = OPEN_COUNT_SQL.replace("CREATE TEMP TABLE open_count
        AS", "")
print("\nQuery plan for the BETWEEN join (the lines about ev):")
for row in conn.execute("EXPLAIN QUERY PLAN" + BETWEEN_PLAN, PARAMS):
    if "ev" in row[3]:
        print("  ", row[3])
t0 = time.perf_counter()
conn.execute("DROP TABLE temp.open_count")
conn.execute(OPEN_COUNT_SQL, PARAMS)
t_between = time.perf_counter() - t0
t0 = time.perf_counter()
conn.execute("DROP TABLE temp.tagged")
conn.execute(TAGGED_SQL, PARAMS)
t_asof = time.perf_counter() - t0
print(f"\nBETWEEN join, count of open episodes:  {t_between *
        1000:7.1f} ms")
print(f"as-of lookup + count (both steps):      {t_asof * 1000:7.1f}
        ms")
```

Reading — Week 2

The lessons first, then these for exact behavior.

- SQLite joins (SELECT, the FROM clause)
- SQLite CASE expressions
- SQLite CREATE VIEW
- DB Browser: importing CSV
- TimescaleDB time_bucket